10

BÖLÜM: KUBERNETES ILE TANIŞMA – ORKESTRA ŞEFI | MODULE: INTRODUCTION TO KUBERNETES – THE CONDUCTOR

Bildirici (Declarative) Model vs Emirsel (Imperative) Model

Declarative Model vs Imperative Model: Desired State Philosophy

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Emirsel Model (Imperative): Garsona 'Mutfaktan bardağı al, 3 adım yürü, musluğu aç, suyu doldur, bana getir' demek gibidir. Bir adım aksarsa her şey kilitlenir. Bildirici Model (Declarative) ise şudur: 'Masanın üzerinde HER ZAMAN 3 bardak su istiyorum!' Biri bardağı kırarsa, garson hiç sormadan anında yeni bir bardak su koyar. K8s bildirici modelle çalışır: İstenen Durum (Desired State) = Gerçek Durum (Actual State)!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Imperative Model: Shouting micromanaged commands: 'Walk 5 steps, grab glass, fill water, bring it'. If step 3 fails, the system crashes. Declarative Model: 'I want 3 glasses of water on this table AT ALL TIMES.' If an earthquake shatters a glass, the waiter immediately replaces it without being asked. That is Kubernetes: Desired State equals Actual State!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Kubernetes'in kalbinde 'Reconciliation Loop' (Mutabakat Döngüsü) yatar. Siz bir YAML manifestosu verdiğinizde bu 'Desired State' (İstenen Durum) olarak etcd'ye yazılır. İlgili kontrolcü (örneğin DeploymentController) durmaksızın şu döngüyü çalıştırır: 1) Observe (Mevcut durumu oku: kaç pod çalışıyor?), 2) Diff (İstenen ile mevcut arasındaki farkı bul: 3 istenmiş, 2 var!), 3) Act (Farkı kapat: 1 yeni pod oluştur). Bu sayede dışarıdan bir pod manuel silinse bile K8s saniyeler içinde farkı onarır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Kubernetes operates via an infinite 'Reconciliation Loop'. You declare the Desired State in YAML, stored in etcd. Controllers execute a continuous 3-step loop: 1) Observe actual cluster state, 2) Diff actual against desired state, 3) Actuate corrective API calls to converge actual state back to desired state. Self-healing is an inherent byproduct of this architecture.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# Emirsel Komut (Tavsiye Edilmez - İz bırakmaz):
kubectl run my-coffee-pod --image=ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0

# Bildirici Manifest (Kurumsal Standart - Git'e konabilir):
# coffee-deployment.yaml dosyasını uygula:
kubectl apply -f coffee-deployment.yaml

# K8s istenen durum ile mevcut durum arasındaki farkı anında kapatır!
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Canlı kümede doğrudan `kubectl edit` veya emirsel komutlarla değişiklik yaparsanız, bir sonraki GitOps veya `kubectl apply` çalışmasında bu manuel değişiklikler ezilir ve kaybolur!

🇬🇧 WATCH OUT

Making ad-hoc imperative edits directly in production leads to configuration drift; the next automated pipeline run will overwrite your changes.

🎯 Özet Çıkarımlar

- Bildirici modelde 'nasıl yapılacağını' değil, 'neyin olması gerektiğini' söyleriz.
- Reconciliation döngüsü 7/24 istenen durum ile gerçek durumu eşitler.
- Tüm altyapı YAML dosyaları olarak Git'te versiyonlanabilir (Altyapı Kod Olarak - IaC).

🎯 Key Takeaways

- In declarative systems, you describe WHAT the state should be, not HOW to get there.
- The reconciliation loop runs 24/7, continuously eliminating drift.
- Declarative YAML manifests enable version-controlled Infrastructure as Code (IaC).